Q1 **Food Delivery Order Processing System** 50 marks

A food delivery platform receives orders simultaneously from customers across multiple locations. Each order requires restaurant verification, menu validation, price calculation, delivery assignment, and order confirmation. Invalid menu items, unavailable food, incorrect quantities, invalid customer details, and unexpected errors should not terminate the application. Multiple orders should be processed using independent threads.

Develop a Java application to:

a) Create classes for Customer, Restaurant, FoodItem, Order, and Delivery.

b) Validate customer and food item information.

c) Handle appropriate built-in exceptions.

d) Create user-defined exceptions such as FoodUnavailableException, InvalidQuantityException, and InvalidOrderException.

e) Use try-catch-finally for order processing.

f) Create multiple threads to process customer orders concurrently.

g) Synchronize access to restaurant inventory when multiple orders contain the same food item.

h) Assign different priorities to express and normal delivery orders.

i) Display successfully processed orders, rejected orders, and remaining food inventory.

Your Code

```
/*a) create classes for customer, restaurant,foodItem,order and delive
A class is a blueprint for creating objects in java. it contains data
and methods that represent the properties and behaviour of an entity
class customer{
    private int customerid;
    private String name;
    private String phone;
    private String address;
    public customer(int customerid,String name,String phone,String add
        this.customerid=customerid;
        this.name=name;
        this.phone=phone;
        this.address=address;
    }
}
class fooditem{
    String name;
    double price;
    fooditem(Srtring name,double price){
        this.name=name;
        this.price=price;
    }
}
class restaurant{
    String name;
    Map<String,Integer>inventory=new HashMap<>();
    restaurant(String name){
        this.name=name;
    }
}
class order{
    int orderid;
    customer customer;
    restaurant restaurant;
    String fooditem;
```

```
    int quantity;
    boolean express;
    order(int id, customer c,restaurant r,String food,int qty,boolean
        orderid=id;
        customer=c;
        restaurant=r;
        foodname=food;
        quantity=qty;
        this.express=express;
    }
}
class delivery{
    String address;
    String type;
    delivery(String address,String type){
        this.address=address;
        this.type=type;
    }
}*/


/*b)validate customer and food item information

validation checks whether the given input satisfies the required condi
it prevents invalid customer detaills,quantities and unavailable food
if(!customer.phone.matches("\\d{10}"))
    throw new IllegalArgumentException("invalid phone");
if(quantity<=0)
    throw new InvalidQuantityException("invalid quantity");
if(!restaurant.inventory.containsKey(foodname))
    throw new FoodUnavailableException("food unavailable");
*/


/*c)handle appropriate built in exceptions

built in exceptions are predefined by java for handling common errors
```

```
if(price<=0)
    throw new IllegalArgumentException("invalid price");
catch(IllegalArgumentException e){
    System.out.println(e.getMessage());
}*/


/*d) create user-defined exceptions such as FoodUnavailableException,I
and InvalidOrderException

user defined exceptions are custom exceptions created by the programme
they are useful for representing application specific errors

class FoodUnavailableException extends Exception{
    FoodUnavailableException(String msg){
        super(msg);
    }
}
class InvalidQuantityException extends Exception{
    InvalidQuantityException(String msg){
        super(msg);
    }
}
class InvalidOrderException extends Exception{
    InvalidOrderException(String msg){
        super(msg);
    }
}*/


/*e)use try-catch finally for order processing

try coontains the code that may cause an exception while catch handles
finally executes whether an exception occurs or not

try{
    System.out.println("Order processed");
```

```
110 }
111 catch(FoodUnavailableException e){
112     System.out.prinltn("food unavailable");
113 }
114 catch(InvalidQuantityException e){
115     System.out.println("invalid quantity");
116 }
117 finally{
118     System.out.println("order processing completed");
119 }*/
120
121
122 /*f) create multiple threads toprocess customer orders concurrently
123 a thread is an independent path of execuion within a program
124 class orderprocessor implements runnable{
125     order order;
126     orderprocessor(oreder order){
127         this.order=order;
128     }
129     public void run(){
130
131     }
132 }
133  Thread t1= new Thread(new ordeprocessor(order1));
134 Thread t2=new thread(ordderprocessor(order2));
135 t1.start();
136 t2.start();*/
137
138
139 /*g)Synchronize access torestaurant inventory whenmultiple orders cont
140
141 synchronization prevents multiple threads form
142 modifying the shared data simultaneously
143
144 public synchronize void updateinventory(String food, int quantity)
145     throws FoodUnavailableException{
146
```

```
146
147     int availabe=nventory.get(food);
148     if(availble<=quantity)
149         throw new FoodUnavailbleException("food unavailable");
150     inventory.put(food,available-quantity);
151
152 }*/
153
154
155 /*h)assign different prioroities to assessan normal delivery orders
156 thread priority indicates relative importance of a thread to the sched
157
158 expressThread.setPriority(Thread.MAX_PRIORITY);
159 normalThreadd.setPriority(Thread.NORM_PRIORITY);
160
161
162 i)display successfully processed orders and rejects orders  and remain
163 List<order> successful=Collections.synchronizedList(new ArrayList<>())
164
165 List<order> rejected=Collections.synchronizedList(new ArrayList<>());
166 System.out.prinltn("successful: "+successful);
167 System.out.println("rejected orders: "+rejected);
168
169 restaurant.displayInventory();
170 */
171
172 //completed code
173
174 import java.util.*;
175 public class fooddeliverysystem{
176     public static void main(String[] args){
177         restaurant r=new restaurant("food hub");
178         r.add("pizza",3);
179         r.add("burger",5);
180         customer c1=new customer(1,"rahul","876974682","banglore");
181         customer c2= new customer(2,"priya","647336281","chennai");
182         order o1=new order(101,c1,r,"pizza",2,true);
```

```
        order o2=new order(102,c2,r,"burger",1,false);
        order o3= new order(103,c1,r,"pizza",5,false);
        Thread t1=new Thread(new processor(o1));
        Thread t2=new Thread(new processor(o2));
        Thread t3=new Thread(new processor(o3));
        t1.setPriority(Threadd.MAX_PRIORITY);
        t2.setPriority(Thread.NORM_PRIORITY);
        t3.setPriority(Thread.NORM_PRIORITY);
        t1.start();
        t2.start();
        t3.start();
        t1.join();
        t2.join();
        t3.join();
        r.showinventory();
    }
}
class customer{
    int id;
    String name,phone,address;
    customer(int id,String name,String phone,String address){
        this.id=id;
        this.name=name;
        this.phone=phone;
        this.adress=address;
    }
    void validate(){
        if(!phone.matches("\\d{10}"))
            throw new IllegalArgumentException("invalid operation");

    }
}
class fooditem{
    String name;
    double price;
    fooditem(String name,double price){
        this.name=name;
```

```
220         this.price=price;
221
222     }
223 }
224 class prprocessor implements runnable{
225     order o;
226     public void run(){
227         try{
228             o.c.valiate();
229             o.r.reserve(o.food,o.qauntity);
230             System.out.println("order"+ oid+" processed "+(o.express?'
231         }
232         catch (FoodUnavailableException e){
233             System.out.println("order"+o.id+"rejected"+e.etMessage()),
234         }
235         catch(InvalidQuantityexception e){
236             System.out.println("order"+o.id+"rejected"+g.getMessage()
237         }
238         catch(IllegalArgumentException e){
239             System.out.println("order"+o.id+"rejected:"+g.getMessage(
240         }
241         finally{
242             System.out.println("finished order"+o.id);
243         }
244
245     }
246 }
247 class FoodUnavailableException extends Exception{
248     FoodUnavailableException(String msg){
249         super(msg);
250     }
251 }
252 class InvalidQuantityException extends Exception{
253     InvalidQuantityException(String msg){
254         super(msg);
255     }
256 }
```